

Computing Through Tools Decorrelates Same-Model LLM Ensembles

Reda Louriki

reda.louriki-etu@etu.univh2c.ma

Abstract

Agents built on the same large language model (LLM) share weights, so repeated sampling or persona prompts tend to produce correlated errors, which limits the benefit of ensembling them. We study how to induce useful error diversity among same-model agents and when that diversity compensates for the individual accuracy it costs. Applying the ambiguity decomposition to the vote distribution of an agent ensemble gives an exchange-rate diagnostic that is exact for the Brier score of the vote distribution; whether it also predicts majority-vote outcomes is tested empirically. On recent competition mathematics problems from the MathArena stream, we compare five ways of diversifying five agents of the same model: repeated sampling, persona prompts, fixed method templates, planner-generated plans, and computational substrates, in which agents solve in plain text, executed Python, SymPy, brute-force search, or solve-then-check code. Pooled across four models and 288 paired items, substrate ensembles improve majority-vote accuracy over self-consistency by +8.3 points (95% CI [+3.7,+12.8], Holm-adjusted $p = 0.002$, random effects +6.5 [+0.1,+12.9]) without generating more tokens. A homogeneous control on one model shows that five copies of the best single tool perform as well as the mixed ensemble ($\Delta = 0.0$ points, 95% CI [-4.4,+4.9]), and that five identical Python agents are much less error-correlated than five text agents. The gain therefore comes mainly from computing through tools rather than from mixing them, although a mixed ensemble reaches the accuracy of the best tool without requiring that tool to be identified in advance. The effect depends on capability. Tools raise per-agent accuracy on weak models and mainly decorrelate errors on stronger ones, and the gain decreases as base accuracy rises (+14.5 $\rightarrow$ +12.5 $\rightarrow$ +3.3 $\rightarrow$ +0.5 points from the weakest to the strongest model). Planner-generated plans reduce individual accuracy without a significant ensemble benefit, and the diagnostic anticipates this out of sample.

1 Introduction

The error of an averaged predictor equals the average member error minus a diversity term [Krogh and Vedelsby, 1994, Wood et al., 2023]. For LLMs the simplest ensemble is self-consistency, which samples one model several times and takes a majority vote [Wang et al., 2023]. Its members share weights and training data, and their errors are strongly correlated. A recent capability-controlled audit reports that common diversity metrics for LLM ensembles mostly track member weakness and that majority vote rarely beats the strongest member [Audit, 2026], and mixing different models can underperform aggregating a single strong one [Li et al., 2025].

The audit examines whether measured diversity predicts ensemble gains. We examine how useful diversity can be induced among agents of the same model, and when it pays for the individual accuracy it costs. Under a fixed budget of five agents per item, we compare stochastic sampling, surface-level prompt variation, procedural variation through method templates, model-generated variation through a planner, and substrate variation, where the same model computes through

different tools. Combining chain-of-thought and program-based reasoning has been studied before. XoT switches between them on verification failure [Liu et al., 2023], and collaborative verification combines them with learned verifiers [Liang et al., 2024]. Here we treat substrates as a source of ensemble diversity, quantify the trade-off between the diversity they add and the accuracy they cost, separate that diversity from tool augmentation with homogeneous-substrate controls, and compare substrates with the other mechanisms at equal budget.

Contributions.

- An application of ensemble ambiguity theory to LLM agent ensembles on the simplex of answer-equivalence classes, giving an exchange-rate diagnostic that is exact for the Brier score of the vote distribution and is tested empirically on majority vote (Section 3).
- A controlled comparison of five diversity mechanisms at a fixed agent budget on recent competition problems, with Holm-corrected, model-stratified and random-effects analyses.
- Evidence across four models that tool-substrate ensembles improve over self-consistency without generating more tokens, and homogeneous controls indicating that the gain comes mainly from computing through tools rather than from heterogeneity itself.
- A capability-dependent mechanism (augmentation for weak models, decorrelation for stronger ones) and a negative result for planner-generated plans.

2 Related work

Sampling-based ensembles. Self-consistency [Wang et al., 2023] and universal self-consistency [Chen et al., 2023] aggregate samples of one model. Mixture-of-Agents layers aggregators over several models [Wang et al., 2025], while same-model aggregation can perform better [Li et al., 2025].

Prompt diversity. DIV-SE elicits and ensembles diverse reasoning approaches [Naik et al., 2023], and Dipper ensembles diverse prompts [Dipper, 2024]. We test both fixed templates and planner-generated approaches while controlling for individual accuracy.

Programs and tools. Program-aided reasoning offloads computation to an interpreter [Gao et al., 2023, Chen et al., 2022]. XoT integrates chain-, program- and equation-of-thought with verification-based switching [Liu et al., 2023], and collaborative verification combines CoT and PoT with verifiers [Liang et al., 2024]. We study such substrates as ensemble members and measure their contribution to diversity separately from their contribution to accuracy.

Ensemble diversity theory. Our accounting instantiates known decompositions [Krogh and Vedelsby, 1994, Wood et al., 2023]. Generalized ambiguity decompositions exist for classification [Jiang et al., 2017], and 0/1 loss has no equally clean additive form [Wood et al., 2023], which is why our diagnostic is stated for the Brier score of the vote distribution. Pairwise diversity measures follow Kuncheva and Whitaker [2003].

3 Framework

Let K agents answer an item with gold answer y , with answers mapped to equivalence classes as described in Section 4. Let $p_i \in \{0, 1\}^M$ be the one-hot class of agent i , $\bar{p} = \frac{1}{K} \sum_i p_i$ the vote distribution, and e_y the gold class. Then

$$\underbrace{\frac{1}{K} \sum_i \|p_i - e_y\|^2}_{\text{mean individual error}} = \underbrace{\|\bar{p} - e_y\|^2}_{\text{Brier error of the vote distribution}} + \underbrace{\frac{1}{K} \sum_i \|p_i - \bar{p}\|^2}_{\text{ambiguity}}. \quad (1)$$

Since $\|p_i - e_y\|^2 = 2 \cdot \mathbb{I}[\text{agent } i \text{ wrong}]$, a diversity mechanism that lowers mean accuracy by δ and raises ambiguity by α improves the Brier error of the vote distribution if and only if $\alpha > 2\delta$. We call $r = \alpha/(2\delta)$ the exchange rate, and say that a mechanism pays when $r > 1$ or $\delta \leq 0$.

Majority vote is the argmax of $\bar{p}$, and 0/1 loss admits no equally clean additive decomposition [Wood et al., 2023]. Whether r predicts majority-vote gains is therefore an empirical question, which we test out of sample in Section 5.8. We also report the mean pairwise error correlation $\bar{\phi}$ and the effective number of independent agents $K/(1 + (K - 1)\bar{\phi})$.

4 Experimental setup

Data. We use final-answer problems from AIME 2025 and 2026, HMMT (February 2025, November 2025, February 2026), SMT 2025, CMIMC 2025 and BRUMO 2025, taken from the MathArena evaluation stream [Balunović et al., 2025]. APEX is excluded because it is close to unsolvable by design. All analyses use one fixed subset of 90 items. Contamination risk varies by model and competition date, and gemma-4 was released after part of the 2025 competitions. Older benchmarks were rejected after a probe: MATH levels 4 and 5 are at ceiling for these models (94 to 100%), and AIME 1983 to 2024 shows signs of memorisation (90% accuracy with about 1.7k output tokens).

Answer equivalence. Final answers are read from the last `\boxed{\}` for text agents and from the last `ANSWER:` line printed by the program for code agents. Two answers belong to the same class if their normalised strings coincide (after removing `\left` and `\right`, `\text{\}`, degree signs, spaces and trailing periods, and mapping `*I` to `i`), if `math-verify` finds them symbolically equivalent in either direction after LaTeX and expression extraction, or, for plain program output such as `sqrt(12)`, `pi` or tuples, if the answer converted to LaTeX with SymPy (unevaluated `sympify`) is found equivalent. Missing or unparseable answers abstain and do not vote. Classes are built jointly over all answers to an item, including the gold answer, so every system and aggregator votes over the same labels. All code is released.

Models. We use gemma-4-31B-it (non-reasoning, NVIDIA NIM), gemini-3.5-flash-lite and gemini-3.1-flash-lite (Google AI Studio), and gpt-oss-20b (reasoning, served by NVIDIA NIM and by Groq on disjoint items, so the A and S agents of any item share a provider). Temperature is 0.7 and the output budget is 16k tokens.

Arms. Each arm has five agents of the same model. **A** uses an identical neutral prompt. **C** uses five fixed method templates. In **G**, a planner writes five problem-specific plans and each solver follows one of them. **S** uses five substrates: text chain-of-thought, executed Python, SymPy formalisation, brute-force or numeric search, and solve-then-check. Code runs in a network-isolated sandbox with one repair round. **X** adds five more neutral samples, so that A and X together form

ten-sample self-consistency. **HB** and **HP** are homogeneous controls with five brute-force or five Python agents.

Aggregation and statistics. We use majority vote over equivalence classes, with ties counted at their expected value, cross-fitted weighted votes, and a verifier in which the base model must pick one of the candidates. Within a model we use a paired item-level bootstrap. Pooled contrasts use a model-stratified sign-flip permutation test, a stratified bootstrap and DerSimonian and Laird random effects, with Holm correction over the planned family $\{S-A, C-A, G-A\}$. Choices made after seeing a model’s data, such as removing SymPy (“S4”), are validated only on the other models. Each item has a single sampled ensemble per arm, so the item-level bootstrap already includes sampling noise; Section 5.7 reports a run-to-run replicate.

5 Results

5.1 Pilot on GSM8K

On GSM8K with llama-3.2-11B (100 items), a blind judge recovered the assigned method in 79% of template-arm solutions, against 17% by chance. Templates halved the error correlation ($\bar{\phi}$ from 0.49 to 0.26) but cost 9.6 points of individual accuracy, an exchange rate of 0.66, and majority-vote accuracy did not change. Persona prompts lowered accuracy without a significant reduction in error correlation.

5.2 Main comparison on gemma-4-31B

Arm	Indiv. acc	Majority	Verifier	Oracle	$\bar{\phi}$	Eff. N
A self-consistency	0.800	0.867	0.889	0.922	0.575	1.52
C fixed templates	0.793	0.867	0.900	0.956	0.443	1.80
G generated plans	0.716	0.833	0.822	0.900	0.459	1.76
S substrates	0.816	0.900	0.922	0.967	0.392	1.95

Table 1: gemma-4-31B, five agents per arm, the same 90 items in every arm.

Relative to A, the exchange rate is 3.1 for C and 0.56 for G, and S loses no individual accuracy.

5.3 Homogeneous-substrate controls

The S arm changes which tool each agent uses and also varies the tool across agents. To separate these two factors we replicate a single substrate five times on the same items, using brute force, the best single substrate on this model, and Python, the most common one. $S-HB = 0.0$ $[-4.4, +4.9]$

gemma-4-31B, $n=81$	Indiv. acc	Majority	Oracle	$\bar{\phi}$
A: text $\times 5$	0.805	0.860	0.914	0.64
HP: Python $\times 5$	0.835	0.898	0.963	0.41
HB: brute force $\times 5$	0.852	0.904	0.938	0.50
S: 5 different substrates	0.820	0.904	0.963	0.40

Table 2: Homogeneous-substrate controls.

($p = 1.000$); S-HP = +0.6 [−2.3, +3.4] ($p = 1.000$). On this model the mixed ensemble performs as well as the best single tool replicated five times, but not better. Five identical Python agents are nearly as decorrelated as the mixed ensemble, so most of the diversity comes from computing through code rather than from mixing tools. The mixed ensemble does not require knowing in advance which tool is best, which matters because the best tool differs across models.

5.4 Pooled analysis across models

Contrast	n	Δ (pts)	95% CI	p_{perm}	p_{Holm}	Random effects	McNemar b/c
S (substrates) – A	288	+8.3	[+3.7, +12.8]	<0.001	0.002	+6.5 [+0.1, +12.9]	39/15
C (templates) – A	141	+1.5	[−3.0, +6.1]	0.549	0.549	−0.7 [−5.8, +4.4]	8/6
G (planner) – A	107	−3.6	[−9.2, +1.7]	0.237	0.475	−3.3 [−7.6, +1.0]	4/7
S4 (no SymPy) – A [†]	288	+8.5	[+3.9, +13.0]	<0.001	n/a	+6.8 [+0.1, +13.4]	37/14

Table 3: Model-stratified pooled contrasts (majority vote; ties at expectation). Holm over the planned family $\{S, C, G\}$. [†]Exploratory.

Per model, S–A is gemma-4-31B $n=90$: +3.3; gemini-3.5-flash-lite $n=67$: +12.5; gemini-3.1-flash-lite $n=84$: +14.5; gpt-oss-20b $n=47$: +0.5. On the models not used to choose it, removing SymPy leaves accuracy unchanged (S4–S = −0.1 [−2.1, +1.9], $p = 0.951$).

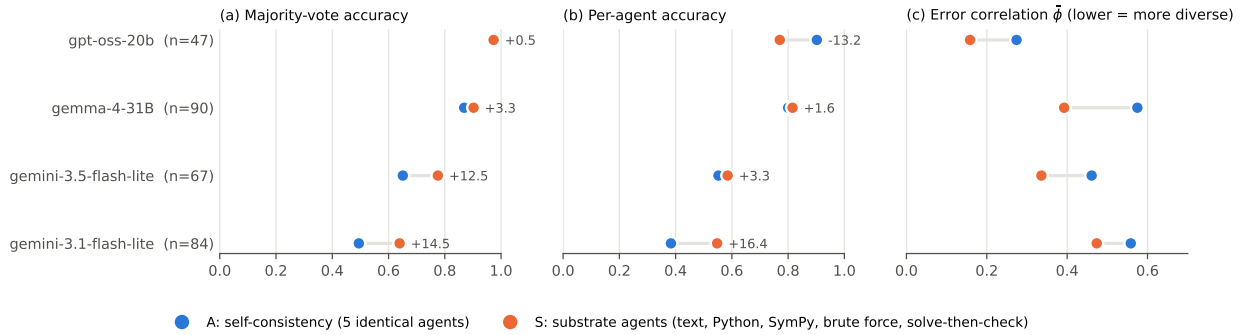


Figure 1: Substrate agents (S, orange) vs. self-consistency (A, blue), per model, ordered by base per-agent accuracy. (a) Majority-vote accuracy; the gain decreases as base accuracy rises. (b) Per-agent accuracy, which tools raise for weak models and lower for the reasoning model. (c) Mean pairwise error correlation, which is lower under S for every model.

Model	n	A tokens/item	S tokens/item	S vs A	Δ majority (pts)
gemini-3.1-flash-lite	84	5.2k	5.7k	+9%	+14.5
gemini-3.5-flash-lite	67	19.8k	12.4k	−37%	+12.5
gemma-4-31B	90	8.2k	9.7k	+18%	+3.3
gpt-oss-20b	47	20.9k	14.7k	−30%	+0.5

Table 4: Output tokens per problem for five agents (S includes code-repair calls). S generates at most 18% more tokens than self-consistency, and fewer on 2 of 4 models, so the gain does not come from additional generation.

5.5 Augmentation and decorrelation

Substrate agents have a lower pairwise error correlation than self-consistency for every model (Figure 1c and Table 5). Their effect on per-agent accuracy depends on capability (Figure 1b). The weaker the model, the more the tools raise the accuracy of each agent. For gpt-oss-20b, the reasoning model and the strongest in our set, tools lower per-agent accuracy, but the ensemble does not lose accuracy because its errors are less correlated, which is the situation the exchange-rate diagnostic describes. Pooled over models, the per-agent gain is +3.9 points [+0.3, +7.5].

Model	n	Per-agent acc (A / S)	$\bar{\phi}$ (A / S)	Majority (A / S)
gemini-3.1-flash-lite	84	0.383 / 0.548	0.56 / 0.47	0.494 / 0.639
gemini-3.5-flash-lite	67	0.552 / 0.585	0.46 / 0.34	0.651 / 0.775
gemma-4-31B	90	0.800 / 0.816	0.57 / 0.39	0.869 / 0.902
gpt-oss-20b	47	0.902 / 0.770	0.27 / 0.16	0.968 / 0.973

Table 5: Per-model decomposition of the S versus A comparison.

5.6 Compute-matched comparison and total cost

System (gemma-4-31B)	Accuracy	Output tokens per problem
A×5, majority vote	0.871	8.1k
A×10 with verifier (compute-matched)	0.894	16.9k
S×5 with verifier	0.918	10.4k

Table 6: Verifier systems on the 85 items for which all verifiers are available.

Table 7 reports total LLM cost per problem for A and S: calls, input tokens and output tokens, averaged over each model’s paired items. Code execution adds 2.7s of sandbox time per problem on gemma (433 executions in total), which is small compared with generation latency.

Model	LLM calls (A / S)	Input tokens (A / S)	Output tokens (A / S)
gemma-4-31B	5.0 / 5.8	0.8k / 3.3k	8.2k / 9.7k
gemini-3.5-flash-lite	5.0 / 6.3	0.7k / 5.1k	19.8k / 12.4k
gemini-3.1-flash-lite	5.0 / 5.6	0.7k / 2.2k	5.1k / 5.6k
gpt-oss-20b	5.2 / 6.4	1.0k / 2.9k	20.9k / 14.7k

Table 7: Total LLM cost per problem (five agents; S includes repair calls).

5.7 Run-to-run stability

On gemma, A and X are two independent five-sample self-consistency ensembles. Their majority-vote accuracies differ by -1.0 points [−7.0, +4.9] over 89 items, and 11% of items change outcome between the two runs, which indicates the size of run-to-run noise for a single model; the pooled analysis over models and items is what gives the main contrast its precision. We also re-ran the S arm independently (S2). The second, independent estimate of the gemma contrast is S2−X = +2.2 [−3.8, +8.5], against S−A = +3.1 [−3.8, +10.0] in the first run. Averaged over both runs, it is +2.7 [−2.1, +7.5] ($n = 80$). Between runs, 9% of items change outcome under S, versus 11% under A.

5.8 Ablation and the exchange-rate diagnostic

Leaving one substrate out on gemma shows the largest contributions from brute-force search (3.3 points lost when removed) and executed Python (2.2 points). Removing SymPy looked beneficial in sample (+1.1 points) but is neutral on held-out models ($S4-S = -0.1$, $p = 0.951$, $n = 207$), so we keep the five-substrate arm. When the exchange rate is estimated on one half of the items and checked on the other, it predicts the failure of generated plans in 99% of random splits for the Brier score, for which it is exact, and in 84% of splits for majority vote. For small gains (below about 3 points with 45 items) its predictions are at chance level. The diagnostic is therefore informative for majority vote, although less reliable than for the quantity from which it is derived.

5.9 Comparison with the MathArena leaderboard

MathArena reports single-attempt accuracy of single models, whereas our systems are multi-agent ensembles at inference time, so the two are not directly comparable and we report output tokens alongside. On the 85 problems shared by all systems, gemma-4-31B with substrate agents and a verifier reaches 0.918 with 10.4k output tokens per problem. On the problems they share with us, o3 (high) scores 0.934 with 9.6k tokens and GPT-OSS-120B (high) scores 0.922. Frontier models remain 5 to 7 points higher.

6 Limitations

Sample sizes are modest, with at most 90 items per model, and gpt-oss-20b is only partially covered, so its results are supporting evidence. The study covers a single domain, competition mathematics, and uses free-tier APIs only. Contamination risk varies by model and competition date. The homogeneous controls were run on one model, so we cannot tell whether heterogeneity helps on weaker models, where per-tool accuracies differ more. Cost accounting covers LLM input and output tokens and sandbox time, and includes hidden reasoning tokens only to the extent that each provider reports them.

7 Conclusion

Letting agents of the same model compute through tools is a practical way to induce useful error diversity. Tools help weak models by raising per-agent accuracy and help stronger models by decorrelating their errors, without increasing the number of generated tokens. On the model where we could test it, mixing different tools did not improve on the best single tool replicated five times; its advantage is that it reaches that level without knowing which tool is best, and the best tool differs across models. Planner-generated diversity did not help in our experiments. An exchange-rate diagnostic derived from the ambiguity decomposition identifies when induced diversity fails to compensate for its accuracy cost.

References

- A. Krogh and J. Vedelsby. Neural network ensembles, cross validation, and active learning. In *Advances in Neural Information Processing Systems 7 (NIPS 1994)*, 1995.
- D. Wood, T. Mu, A. Webb, H. Reeve, M. Luján, and G. Brown. A unified theory of diversity in ensemble learning. *JMLR*, 24(359), 2023.

- Z. Jiang, H. Liu, B. Fu, and Z. Wu. Generalized ambiguity decompositions for classification with applications in active learning and unsupervised ensemble pruning. *AAAI*, 2017.
- L. I. Kuncheva and C. J. Whitaker. Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. *Machine Learning*, 51(2):181-207, 2003.
- X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. *ICLR*, 2023.
- X. Chen et al. Universal self-consistency for large language model generation. arXiv:2311.17311, 2023.
- J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou. Mixture-of-Agents enhances large language model capabilities. *ICLR*, 2025.
- W. Li et al. Rethinking Mixture-of-Agents: is mixing different large language models beneficial? arXiv:2502.00674, 2025.
- R. Naik, V. Chandrasekaran, M. Yuksekgonul, H. Palangi, and B. Nushi. Diversity of thought improves reasoning abilities of LLMs. arXiv:2310.07088, 2023.
- Dipper: Diversity in prompts for producing large language model ensembles in reasoning tasks. arXiv:2412.15238, 2024.
- L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: Program-aided language models. *ICML*, 2023.
- W. Chen, X. Ma, X. Wang, and W. W. Cohen. Program of thoughts prompting: disentangling computation from reasoning for numerical reasoning tasks. *TMLR*, 2023. arXiv:2211.12588.
- T. Liu, Q. Guo, Y. Yang, X. Hu, Y. Zhang, X. Qiu, and Z. Zhang. Plan, verify and switch: integrated reasoning with diverse X-of-thoughts. *EMNLP*, 2023.
- Z. Liang et al. Improving LLM reasoning through scaling inference computation with collaborative verification. arXiv:2410.05318, 2024.
- Are diversity metrics measuring diversity? A capability-controlled audit of majority-vote gain in LLM ensembles. arXiv:2607.20768, 2026.
- M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, and M. Vechev. MathArena: evaluating LLMs on uncontaminated math competitions. arXiv:2505.23281, 2025.